

JOINS

When working with databases, we often store data across several interconnected tables. This helps in reducing data redundancy (repeating data unnecessarily).

However, at certain times, we need to combine information scattered across these multiple tables to obtain a single, unified result. This is where the **JOIN** concept in DBMS becomes crucial.

Put simply, **a JOIN is a method of combining rows from two or more tables based on a common field existing between them.**

Understanding with an Example

Let's imagine we have two tables as shown below:

- **Customers Table:** Contains details of customers.
- **Orders Table:** Contains details of orders placed by them.

These two tables are connected via the common column, `CustomerID`.

Table 1: Customers

CustomerID	CustomerName
1	Amal
2	Kamal
3	Nimal

Table 2: Orders

OrderID	CustomerID	Amount
101	1	5000
102	2	7000
103	4	2000

Note: Here, **CustomerID 3** (Nimal) has not placed any orders.

Also, **CustomerID 4** has placed an order (OrderID 103), but their name is not listed in the Customers table. These differences are important for understanding the types of JOINS.

Let's look at the 4 main types of JOINS.

1. INNER JOIN

An **INNER JOIN** returns only the **matching records** found in both tables. If a row doesn't exist in one of the two tables, it will not be included in the result.

- **Concept (Venn Diagram):** Imagine two circles representing the tables. The INNER JOIN is only the middle section where the two circles intersect.

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
INNER JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result: Since both Amal (1) and Kamal (2) exist in both tables, only their data is retrieved.

CustomerName	OrderID
Amal	101
Kamal	102

2. LEFT JOIN (or LEFT OUTER JOIN)

A **LEFT JOIN** returns **all the data from the table on the left (the first table)**, and only the matching data from the table on the right (the second table). If there is no matching data in the right table, it shows a “NULL” value instead.

- **Concept (Venn Diagram):** The entire left circle, plus the intersecting section of the right circle.

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
LEFT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result: All individuals from the Customers table (Amal, Kamal, Nimal) appear in the result.

CustomerName	OrderID
Amal	101
Kamal	102
Nimal	NULL

3. RIGHT JOIN (or RIGHT OUTER JOIN)

This is the opposite process of the LEFT JOIN. A **RIGHT JOIN** returns **all the data from the table on the right (the second table)**, and only the matching data from the table on the left. If there is no matching data in the left table, it shows a “NULL” value.

- **Concept (Venn Diagram):** The entire right circle, plus the intersecting section of the left circle.

```
SELECT Customers.CustomerName, Orders.OrderID
FROM Customers
RIGHT JOIN Orders ON Customers.CustomerID = Orders.CustomerID;
```

Result: All orders found in the Orders table (101, 102, 103) appear in the result.

CustomerName	OrderID
Amal	101
Kamal	102
NULL	103

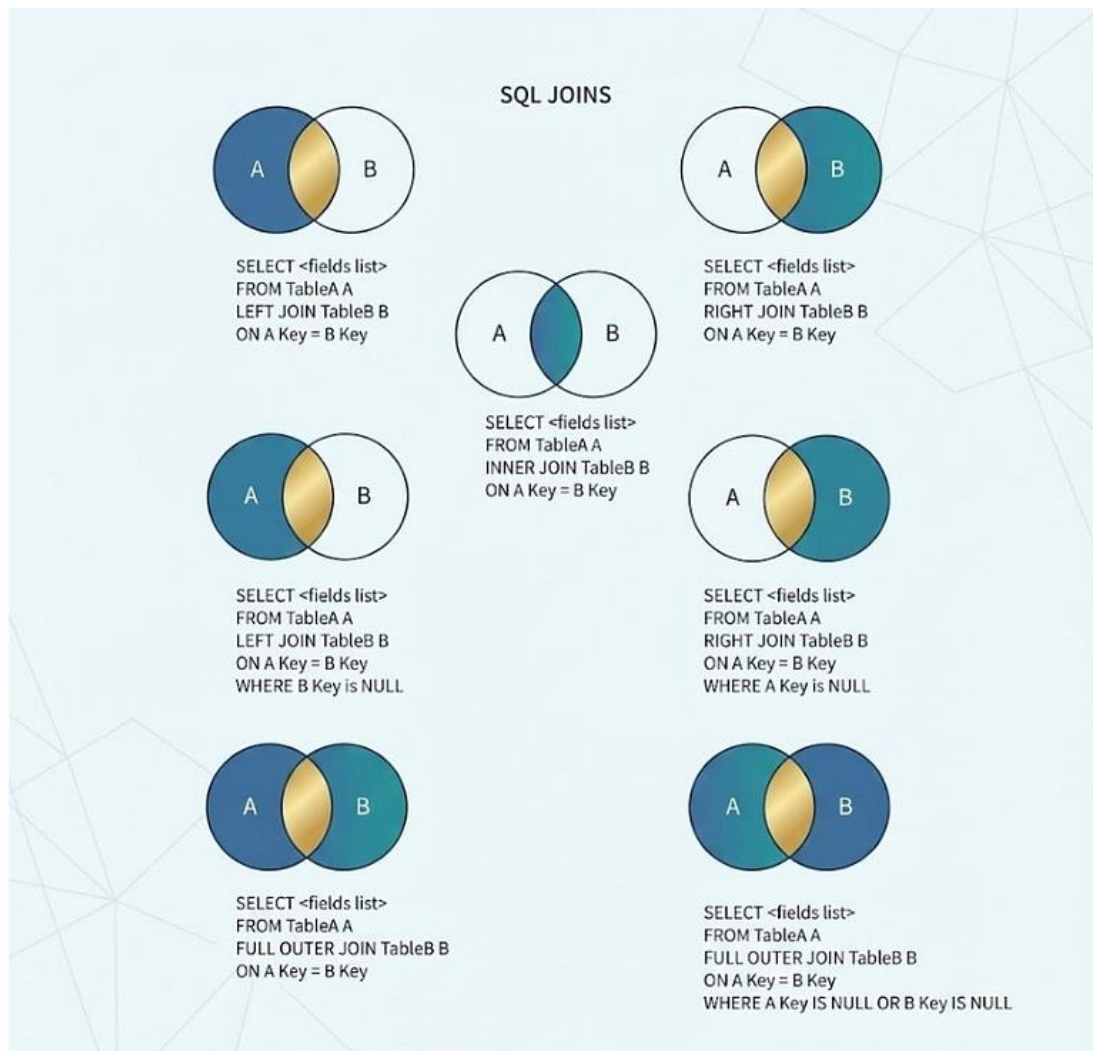
4. FULL JOIN (or FULL OUTER JOIN)

A **FULL JOIN** combines and shows **all data** from both tables. Whenever a connection cannot be established between the two tables, it puts “NULL” values in the relevant places. (This is essentially a combination of a LEFT JOIN and a RIGHT JOIN).

- **Concept (Venn Diagram):** Both circles entirely.

CustomerName	OrderID
Amal	101
Kamal	102
Nimal	NULL
NULL	103

Summary



- **INNER JOIN:** Only the common elements present in both.
- **LEFT JOIN:** The entire left side + matching parts of the right.
- **RIGHT JOIN:** The entire right side + matching parts of the left.
- **FULL JOIN:** Everything present in both tables.